National Tsing Hua University
Department of Electrical Engineering
EE6620 Computational Photography, Spring 2026

Homework #2

Non-Blind Deblurring

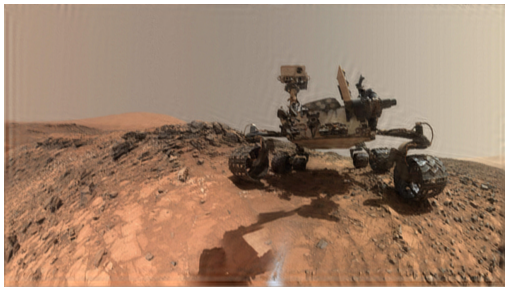
Assigned on April 1, 2026

Due by April 22, 2026

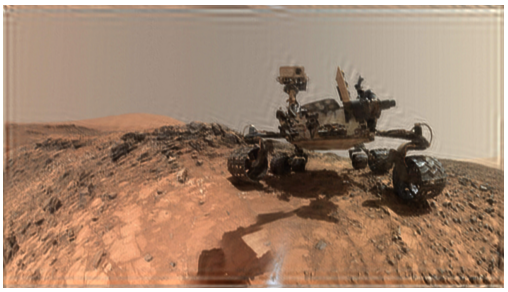
Homework Description

A slow shutter speed will introduce blurred images due to camera shake. The objective of this assignment is to implement several non-blind deblurring algorithms and analyze the effects on blurred images. In part 1-3, you will implement Wiener deconvolution, Richarson-Lucy (RL) deconvolution [1] [2] and its bilateral variant (BRL) [3]. And in part 4, you are going to solve deblurring problem in total variation model [4]. There are two synthetic images in this assignment. The **Curiosity-small** and **Curiosity-meidium** blurred images were synthesized using small and medium blur kernel sizes respectively. For simplicity, the blurring was performed in non-linear ($R'/G'/B'$) domain.

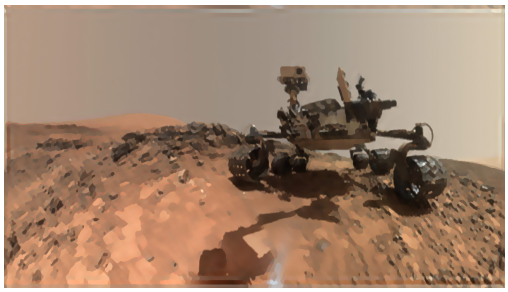
Blurred



Wiener deconv



RL deconv



BRL deconv

Table 1: Scores

Items		Score
Implementation (35%)	Preprocess	5%
	Wiener deconvolution	5%
	RL deconvolution	10%
	BRL deconvolution	10%
	Total variation regularization	5%
Report (65%)	Exploration	35%
	Research study	20%
	Free Study	10%

For coding environment, we are using Google Colab as in the previous homework. You must submit through sharing the entire folder with `vickychen910716@gmail.com` as editor. Do not download and re-upload since coding histories would be checked. **Any work lacking editing history will be flagged for plagiarism or AI generation, leading to a 25% grade penalty.**

1 Implementation (35%)

In this section, you will implement several algorithms for non-blind deblur flows. For each flow, you need to establish in the python codes (deblur_functions.ipynb and TV_functions.ipynb), and the corresponding test functions are provided in (test_deblur_functions.ipynb). In test_deblur_functions.ipynb, $TEST\_PAT\_SIZE$ can be assigned to either `small` or `large`. To speed up development, you could set $TEST\_PAT\_SIZE$ to `small` for quick debugging. **However, you need to pass test functions with `large` setting to get the full score in this section. Additionally, please do not modify the original function I/O. Otherwise, your implementation will be considered as incorrect.**

There are some details you should comply with in implementation:

1. Perform Wiener, RL and BRL deconvolution for the **three color channels separately**.
2. For convolution and padding, the **symmetric border condition** (Appendix: Figure 3) is used in the reference answer. However, you can try some different boundary conditions in your report.
3. RL/BRL can work for different choices of initial images $I^{(0)}$. The blurred image (original image) is a typical choice, but you can try other initial images (for example, a flat black image) in your report.
4. When doing pixel-wise division in RL and BRL deconvolution (i.e., x/y), **code it as x/(y+DBL_MIN) to avoid zero-division issue.**
5. It is allowed to use following library functions to enhance operating efficiency in this assignment. Note that no additional libraries shall be imported for the implementation part.

i `scipy.signal.convolve2d()`

ii numpy-array operation (e.g., `np.exp()`, `np.sum()`, `np.fft.rfft2()`, `np.fft.irfft2()`, etc)

1.1 Preprocess (5%)

Before applying deblur methods, we would need to preprocess the images and kernels. Finish these two functions in deblur_functions.ipynb and use it in the following deconvolution methods.

1. `img_conversion()`: We are using **floating-point** operations, so you shall normalize the array element (e.g., B, I, K) to **[0, 1]** through dividing 255 before performing deblur. After deblurring, you'll need to convert it back to [0,255] (uint8). Be aware of rounding.
2. `kernel_conversion()`: Normalize the blur kernel K such that $\sum k_i = 1$ before using it.

1.2 Wiener deconvolution (5%)

Wiener deconvolution can be express as

$$\hat{I(f)} = \frac{B(f)}{K(f)}\left[\frac{|K(f)|^2}{|K(f)|^2 + \frac{1}{SNR(f)}}\right] \tag{1}$$

$$\hat{I} = IFFT\{\hat{I(f)}\} \tag{2}$$

where

$$B(f) = FFT\{B\}, K(f) = FFT\{K\} \tag{3}$$

To obtain the correct result, you shall first perform some preprocessing steps (Figure 1) before applying DFT to the blur kernel. Finish the function `kernel_preprocess()` in deblur_functions.ipynb.

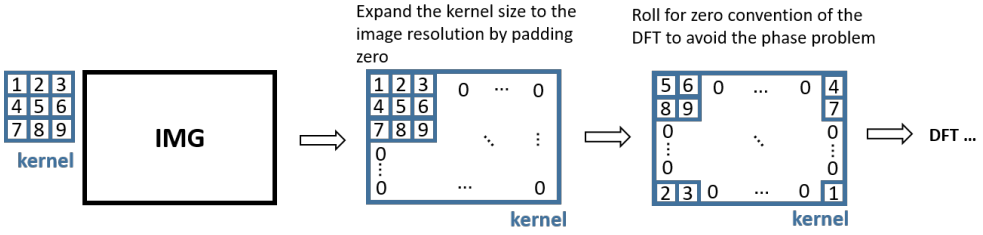


Figure 1: Preprocessing of 2D-DFT on blur kernel

Then, you shall perform Wiener deconvolution in `Wiener()` in deblur_functions.ipynb. You must utilize the functions you just finished, including `img_conversion`, `kernel_conversion` and `kernel_preprocess`. The PSNR should be larger than 60 dB for `Wiener()`.

To get the full score, ensure you pass the Wiener deconvolution test function with large $TEST\_PAT\_SIZE$.

a. Wiener deconvolution on curiosity_medium, with $SNR(f)$ = 100.0.
(tested by `test_Wiener_deconv()`)

1.3 RL deconvolution (10%)

The energy function of RL can be formulated as

$$E(I) = \sum_{ch} \left\{ \sum ((I \otimes K) - B \circ \ln(I \otimes K)) \right\}. \tag{4}$$

Finish the function `check_RL_energy()` in deblur_functions.ipynb to calculate the energy of RL deblurred results. The error for `check_RL_energy()` shall be smaller than 0.05%.

RL deconvolution can be expressed as

$$I^{t+1} = I^t \circ [K^* \otimes \frac{B}{I^t \otimes K}], \tag{5}$$

where

$$K^*(i,j) = K(-i,-j). \tag{6}$$

Finish the function `RL()` in deblur_functions.ipynb. `RL()` shall be able to process the input image by RL deconvolution, and the PSNR shall be larger than 60 dB.

To get the full score, ensure you pass the following RL test functions with large $TEST\_PAT\_SIZE$.

a. Calculate RL energy function.
   (tested by `test_RL_energy()`)
b. RL on curiosity_small, with iteration = 30.
   (tested by `test_RL_a()`)
c. RL on curiosity_medium, with iteration = 45.
   (tested by `test_RL_b()`)

1.4 BRL deconvolution (10%)

The energy function of BRL can be formulated as

$$E(I) = \sum_{ch} \left\{ \sum \{(I \otimes K) - B \circ \ln(I \otimes K)\} + \lambda E_B(I) \right\}, \tag{7}$$

where

$$E_B(I) = \sum_{x} \sum_{y \in \Omega} \exp\left(-\frac{|x-y|^2}{2\sigma_s}\right)\left(1 - \exp\left(-\frac{|I(x) - I(y)|^2}{2\sigma_r}\right)\right) \tag{8}$$

Finish the function `BRL_EB()` and `BRL_energy()` in deblur_functions.ipynb. `BRL_EB()` function calculates Eq. (8) and `BRL_energy()` function calculates the energy of the deblurred results of BRL (Eq. (7)). The error for `BRL_energy()` should be smaller than 0.05% .

BRL deconvolution can be expressed as

$$I^{t+1} = \frac{I^t}{1 + \lambda \cdot \nabla E_B(I^t)} \circ (K^* \otimes \frac{B}{I^t \otimes K}), \tag{9}$$

where

$$K^*(i,j) = K(-i,-j), \tag{10}$$

and

$$\nabla E_B(I^t) = 2 \cdot \sum_{y \in \Omega} (\exp\,(-\frac{|x-y|^2}{2\sigma_s}) \cdot \exp\,(-\frac{|I(x)-I(y)|^2}{2\sigma_r}) \cdot \frac{(I(x)-I(y))}{\sigma_r}). \tag{11}$$

In (11), x is the center pixel and y are nearby pixels in Ω.

Finish the function `BRL()` in deblur_functions.ipynb. `BRL()` function should be able to process the input image by BRL deconvolution, and the PSNR should be larger than 55 dB.

To get the full score, ensure you pass the following BRL test functions with large $TEST\_PAT\_SIZE$.

a. Calculate BRL energy function.
   (tested by `test_BRL_energy()`)
b. BRL on curiosity_small, with
   $\lambda = 0.03/255$, iteration=15, $\sigma_r = 50/255^2$, $r_K = 6$, $r_\Omega = 0.5 r_K$, $\sigma_s = (r_\Omega/3)^2$.
   (tested by `test_BRL_a()`)
c. BRL on curiosity_small, with
   $\lambda = 0.06/255$, iteration=15, $\sigma_r = 50/255^2$, $r_K = 6$, $r_\Omega = 0.5 r_K$, $\sigma_s = (r_\Omega/3)^2$.
   (tested by `test_BRL_b()`)
d. BRL on curiosity_medium, with
   $\lambda = 0.001/255$, iteration=40, $\sigma_r = 25/255^2$, $r_K = 12$, $r_\Omega = 0.5 r_K$, $\sigma_s = (r_\Omega/3)^2$.
   (tested by `test_BRL_c()`)
e. BRL on curiosity_medium,with
   $\lambda = 0.006/255$, iteration=40, $\sigma_r = 25/255^2$, $r_K = 12$, $r_\Omega = 0.5 r_K$, $\sigma_s = (r_\Omega/3)^2$.
   (tested by `test_BRL_d()`)

1.5 Solving the deblurring problem by total variation regularization (5%)

You are going to implement the Total Variation algorithm using ProxImaL[4] in TV_functions.ipynb. An example usage in function `TVL1()` shows how to implement the Total Variation model with norm 1 using ProxImaL. In `TVL2()`, you need to change the data term from norm 1 to norm 2. In `TVpoisson()`, you need to change the data term from norm 1 to model the noise in the blurring process as Poisson distribution (hint: use ProxImaL function `poisson_norm()`).

To get the full score, ensure you pass the following total variation test functions with large $TEST\_PAT\_SIZE$.

a. TVL2 on curiosity_medium, with $\lambda = 0.01$, iteration=1000.
   (tested by `test_TVL2()`)
b. TVpoisson on curiosity_medium, with $\lambda = 0.01$, iteration=1000.
   (tested by `test_TVpoisson()`)

2 Report(65%)

There are three parts of problems: Exploration, Research study, and Free study. Answer the problems clearly and integrally in your report.

2.1 Exploration (35%)

a. A conceptual figure (Figure 2) shows how to evaluate your deblurred result by 1-D DFT scanline. Try to plot a same figure for the deblurred results and a non-blurred image data/curiosity.png. How do you plot the figure? Do you apply any additional steps? Why? What is the meaning of the figure? How can this figure help you explain the results? Does evaluating the results with PSNR reveal the same trend? Explain what you observed. **(10%)**

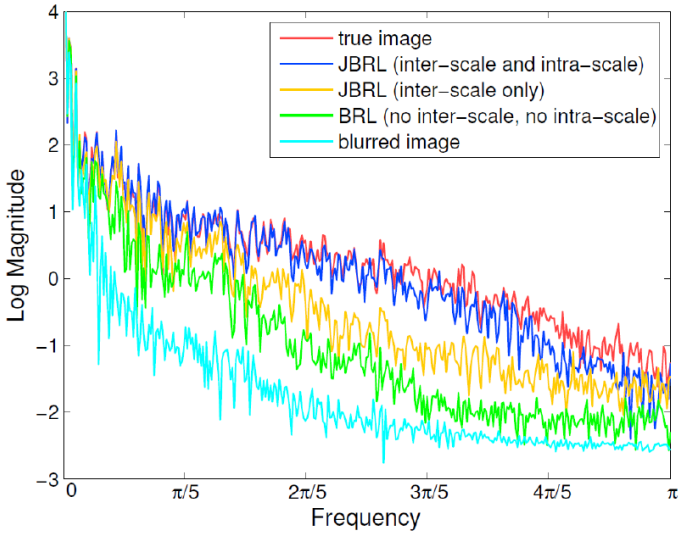


Figure 2: DFT curves of a 1-D scanline

b. Utilizing `check_RL_energy()` and `RL()` in deblur_functions.ipynb, plot how the energy changes throughout the iterations and explain the figure. Use `test_RL_b()` in test_deblur_functions.ipynb as your testcase. **(5%)**

c. BRL requires doing convolution in time domain twice for every iteration, which makes it time-consuming. Check out other `scipy.signal` functions and explore methods to speed it up. Show how much execution time you've saved. **(5%)**

d. Deblur your own blurred images. Photograph two blurred images with different kinds of kernel. Estimate their blur kernels by yourself, and generate your own deblurred images using BRL deconvolution. A reference method to photograph a blur image with a blur kernel is shown in (Appendix: Figure 4). Provide at least the following information in your report. **Notice that when conducting deconvolution to a photo taken in real life, you should transform the photo to the linear domain first (e.g.,**

$img^{2.2}$**), and then transform it back to the nonlinear domain (e.g.,** $img^{\frac{1}{2.2}}$**) after finishing your deblur flow! Using pre-exist datasets from the internet is prohibited. Your student ID card must be presented in the images or no points would be given. (15%)**

i Your blurred images, kernels and result deblurred images

ii How do you extract the kernels? Do you perform an extra preprocessing on your blur kernels or images? Why?

iii How do you choose your parameters? (e.g., λ, iteration, etc)

iv Compare the deblur effect for two different kernels.

2.2 Research Study (20%)

In this section, you should answer the problem by the two-step research flow: **Assumption** and **Justification**. Based on your observation, propose an assumption, and justify your idea by experiments. This research flow may be a loop:

observation $\rightarrow$ **propose assumption** $\rightarrow$ **justification** $\rightarrow$ **modify assumption** $\rightarrow$ **justification ...**

After reaching a satisfactory result, you need to describe your assumption and how you justify it by experiments in the report. Grading criteria for this part are based on clarity of the report, rigorousness of the justification and the overall research flow.

a. When implementing Wiener deconvolution, you need to first expand the kernel size and roll the kernel to get a correct result. Without a rolling kernel, how does it influence the result images? **(5%)**

b. In this assignment, you have done many kinds of non-blind deblurring flows (i.e., Wiener, RL, BRL, and TV). And since it is inevitable to have some noise in the picture, being robust to the noisy image is crucial. Which deblur flow is more robust to the noisy blurred image? **(10%)**

c. Following the previous problem, try modifiying TVL1 parameters to enhance the result. Find the relationship between σ of the noise and the parameters. **(5%)**

2.3 Free Study (10%)

In this section, please propose **one** interesting topic, and answer it by the two-step research flow. Though it's encouraged to propose on your own, you can still choose your free study topic from the following problems. **TA will consider both the breadth and depth of the free study, choosing the top two students. And 2% and 1% bonus scores on semester will be given to the best and the second-best students**

a. Implement another deblurring flow (e.g. multi-scale approach or burst deblurring) and compare to the results in this homework.

b. Accelerate deblurring flow and report the execution time difference.

c. For Wiener deconvolution, establish your flow to find the suitable SNR(f) value of the new blurred images.

d. Compare the BRL results of curiosity-medium with different parameters. Many settings can be varied, such as the ratio λ, σ_r, r_Ω to r_K, etc.

3 Deliverable

1. Submit only your report with filename {studentID}_report.pdf to eeclass.
2. Modify your folder name to HW2_{studentID}, and share with vickychen910716@gmail.com as editor. (Folder structure shall be identical to the assigned homework.)
   (a) Your deblur_functions.ipynb, TV_functions.ipynb and any .ipynb written for the report in folder /code.
   (b) Your result deblurred images of problem 2.1.d in folder /MyDeblur_result
   (c) Images you've taken and the kernels in folder /data
   (d) Your report in Google Doc

Note that editing histories will be reviewed. As mentioned earlier, if no history is found, the work may be considered generated by AI or plagiarized, resulting in a 25% penalty. Additionally, wrong file delivery or arrangement will get 5% punishment.

4 Appendix

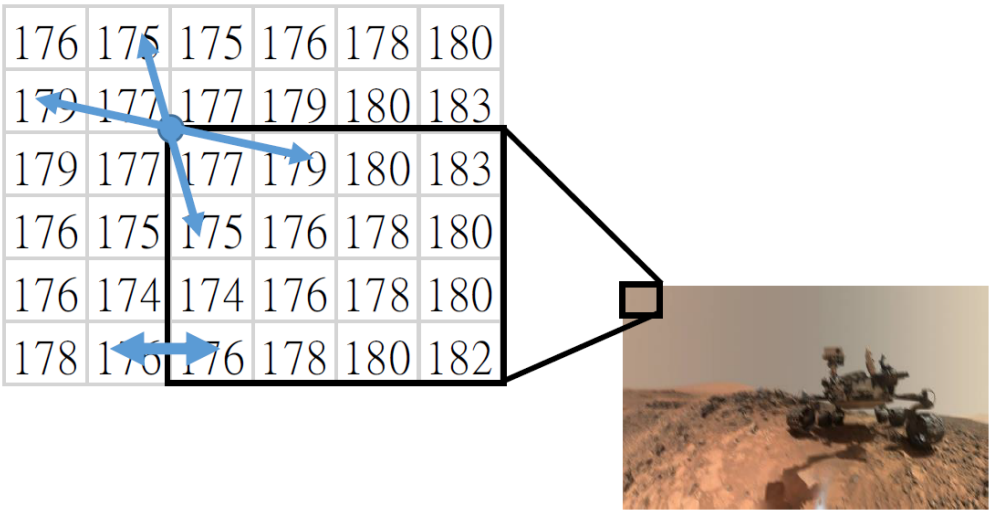


Figure 3: symmetric border condition

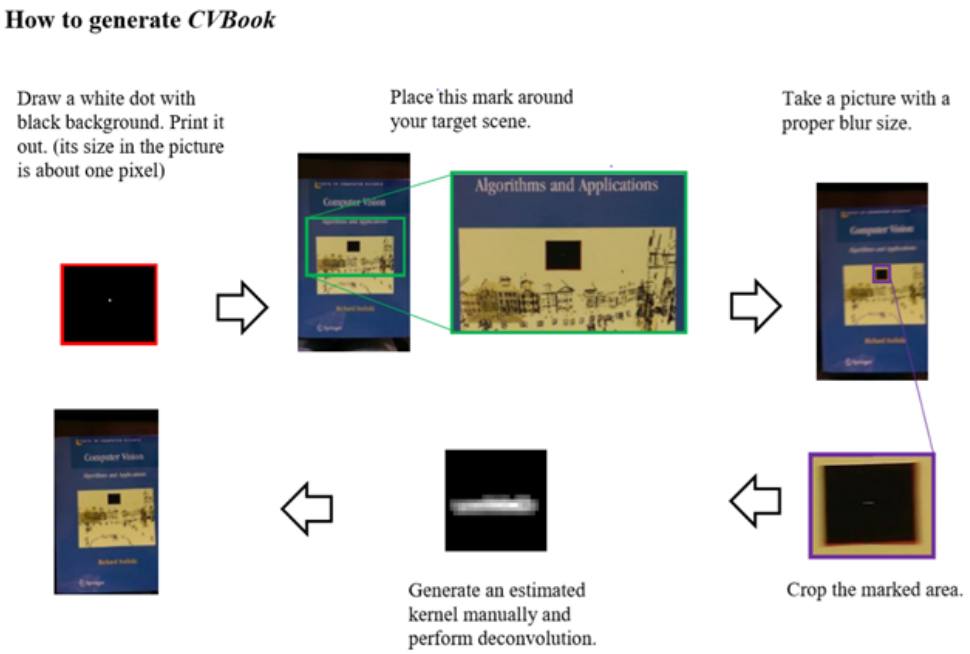


Figure 4: How to generate CVBook

References

[1] W. H. Richardson, "Bayesian-based iterative method of image restoration," vol. 62, no. 1, pp. 55–59, 1972.

[2] L. B. Rucy, "An iterative technique for the rectification of observed distributions," *Astron. J.*, vol. 79, no. 6, pp. 745–754, 1974.

[3] L. L. Yuan, J.Sum and H.-Y. Shum, "Progressive inter-scale and intra-scale non-blind image deconvolution," *ACM Trans. Graph.*, vol. 27, no. 3, pp. 1–10, 2008.

[4] F. Heide, S. Diamond, M. Nießner, J. Ragan-Kelley, W. Heidrich, and G. Wetzstein, "Proximal: Efficient image optimization using proximal algorithms," *ACM Trans. Graph.*, vol. 35, July 2016.